

Day-58

Dynamic Programming (DP)

Fibonacci Series

0 1 1 2 3 5 8 13 21

$$fib(n) = fib(n-1) + fib(n-2)$$

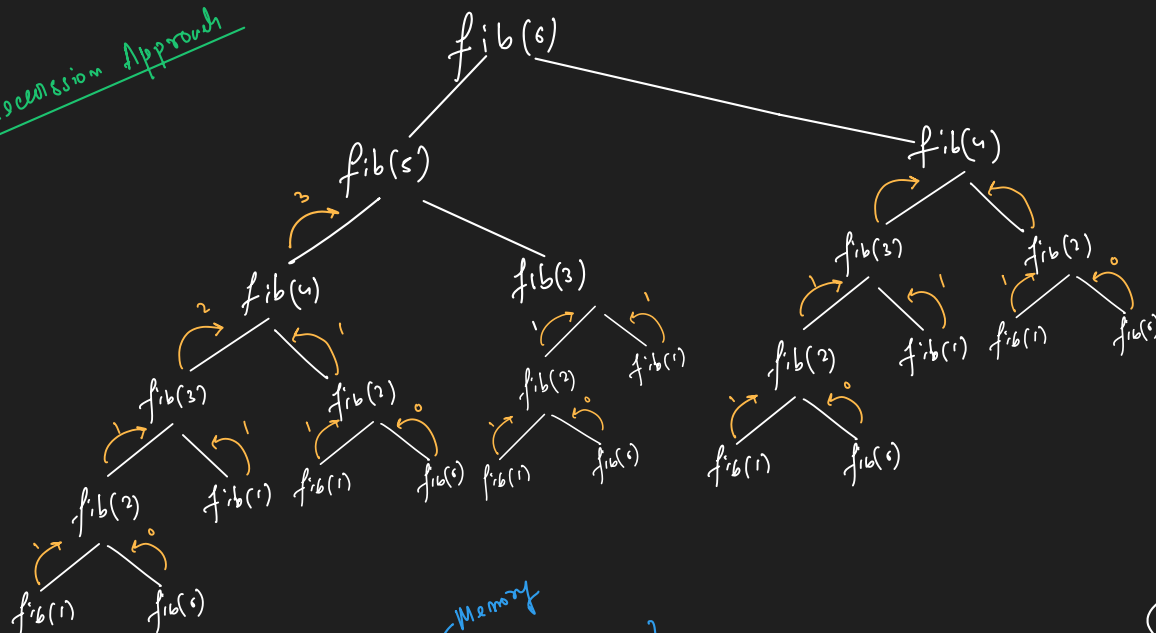
$fib(n)$?

if ($n == 0 || n == 1$) // base case
return n ;

return $fib(n-1) + fib(n-2)$;

}

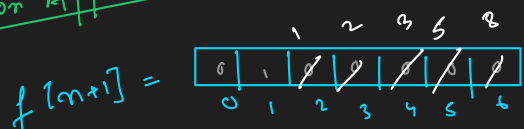
① Recursion Approach



$T.C = (2^n)$

② Memorization Approach

Memory
[Array + Recursion]



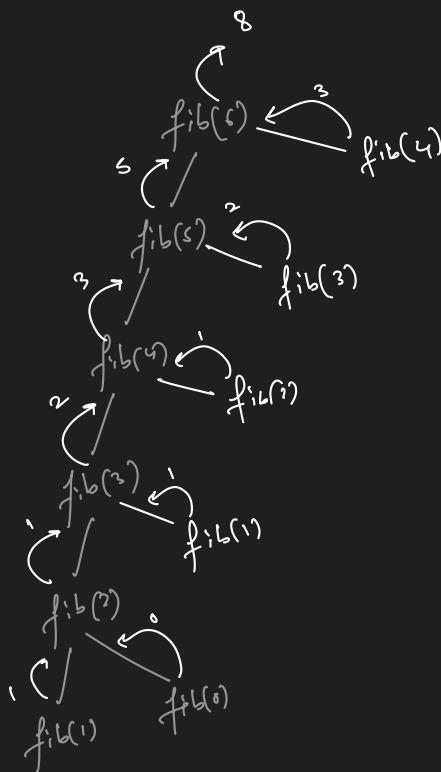
$fib(int n, int[] f)$?

if ($n == 0 || n == 1$) // base case
return $f[n] = n$;

if ($f[n] != 0$) // $fib(n)$ is calculated
return $f[n]$;

memorization $\rightarrow f[n] = fib(n-1, f) + fib(n-2, f)$;
return $f[n]$;

}



$T.C = O(n)$

Top-Down approach

③ Tabulation Method [Array + for loop iteration]

$dp[n+1]$

$dp[1] = 1$

for (int i = 2; i ≤ n; i++) {

$dp[i] = dp[i-1] + dp[i-2];$

}

return $dp[n]$;

T.C = $O(n)$

[Bottom-up approach]

DP : It is a technique in programming that helps to efficiently solve a class of problems that have
(i) overlapping sub-problems and (ii) optional sub-structure property.
(choices) } least, most, min, max, largest, smallest } ← Greedy.